HRMS Attendance Module

Complete Technical, Business, Calculation, API & Testing Documentation

Purpose: One reference for developers, business users, testers and support teams. It documents the current stable Attendance implementation, business rules, workflow, calculations, APIs, dependencies, database requirements and test scenarios.

Latest agreed rule: Approved full-day leave remains **LEAVE**. Approved half-day leave, where LeaveRequest.TotalDays = **0.50**, produces AttendanceRecord.Status = **HalfDay**. No HALF_DAY row is added to AttendanceDayStatuses.

The current architecture keeps attendance calculations centralized in **AttendanceCalculationService**, calendar classification in **AttendanceCalendarBusinessRules / AttendanceDayStatusService**, and persistence behind generic repositories.

1. Architecture & Responsibilities

ProcessAttendanceRecordsCommandHandler orchestrates date-range processing. AttendanceDayStatusService determines the calendar classification. AttendanceCalendarBusinessRules checks weekly off, holidays and approved leave. AttendanceCalculationService calculates worked time and attendance outcomes for working days. Manual Create/Update handlers resolve assignment, shift and policy and use the centralized calculation service.

Leave remains the source of truth for leave duration. Attendance reads LeaveRequest.TotalDays; it does not recalculate leave duration.

2. End-to-End Workflow

1. Receive EmployeeId, FromDate and ToDate. **2.** Load raw logs, applicable shift assignments, shifts and policies. **3.** Iterate each date. **4.** Skip if no assignment or valid shift/policy. **5.** Skip if a non-deleted AttendanceRecord already exists. **6.** Determine day status. **7.** Weekly off/holiday/full-day leave become calendar records. **8.** Half-day approved leave (TotalDays 0.50) becomes an AttendanceRecord with Status HalfDay. **9.** Working days use raw IN/OUT logs. **10.** No punches produce Absent. **11.** Otherwise centralized calculation runs. **12.** Record is persisted.

Half-day path: **Approved Leave** → **TotalDays 0.50** → **day status LEAVE + LeaveDays 0.50** → **processor writes Status HalfDay**. Full-day path remains **TotalDays 1.00** → **LEAVE**.

3. Calendar Day Status Business Rules

Code	Rule / Meaning
WORKING_DAY	Not weekly off, not company holiday, and no approved leave.
WEEKLY_OFF	Saturday or Sunday under the current AttendanceCalendarBusinessRules.
HOLIDAY	Active, non-deleted CompanyHoliday matching the date.
LEAVE	Active, non-deleted LeaveRequest with APPROVED status covering the date.
HalfDay outcome	Not a calendar master code. Approved LeaveRequest.TotalDays = 0.50 is translated by processing into AttendanceRecord.Status = HalfDay.

4. Attendance Calculation Logic

The calculation logic is centralized so Create/Update/Process do not maintain separate formulas.

4.1 Reset

WorkedMinutes, LateMinutes, EarlyLeaveMinutes and OvertimeMinutes are reset to zero before calculation. This prevents stale derived values when an existing record is recalculated.

4.2 Punch validation

No CheckIn → **MissingIn**. No CheckOut → **MissingOut**. CheckOut <= CheckIn → **MissingOut**.

4.3 Worked minutes

WorkedMinutes = elapsed minutes between CheckIn and CheckOut − **Shift.BreakMinutes**, with a minimum of zero. Example: 09:00 to 18:00 = 540 elapsed minutes; with a 60-minute break = 480 worked minutes.

4.4 Scheduled shift

ScheduledStart = AttendanceDate + Shift.StartTime. ScheduledEnd = AttendanceDate + Shift.EndTime. If IsOvernight and EndTime <= StartTime, ScheduledEnd is moved to the next day.

4.5 Late minutes

LateThreshold = ScheduledStart + Policy.GracePeriodMinutes. If CheckIn is later than the threshold, LateMinutes is actual CheckIn minus ScheduledStart. Grace therefore prevents small lateness from being flagged.

4.6 Early leave

For non-overnight shifts, if CheckOut is before ScheduledEnd, EarlyLeaveMinutes = ScheduledEnd – CheckOut. The current implementation does not apply this early-leave calculation to overnight shifts.

4.7 Overtime

Only when Policy.IsOvertimeAllowed and WorkedMinutes > Policy.FullDayMinutes. Raw overtime = WorkedMinutes – FullDayMinutes. It is awarded only when it meets MinimumOvertimeMinutes and is capped at MaximumOvertimeMinutes.

4.8 Status determination

Current centralized logic: WorkedMinutes == 0 → Absent; WorkedMinutes below the implemented half-day threshold → HalfDay; Late + Overtime → LateOvertime; Late → Late; EarlyLeave → EarlyLeave; Overtime → Overtime; otherwise → Present. Keep the half-day threshold aligned with the approved AttendancePolicy/business definition.

4.9 Important distinction

A leave-driven HalfDay and a punch-driven HalfDay are both AttendanceRecord.Status = HalfDay, but their business causes differ. The new half-day leave path requires no punches because Option 1 was selected: approved 0.50 leave itself determines HalfDay.

5. Attendance Status Types

Status	Meaning
Present	Valid attendance with no late, early-leave or overtime status taking precedence.
Absent	Working day with no worked minutes / no attendance punches.
HalfDay	Attendance is treated as half day; includes the newly supported approved 0.50-day leave path.
Late	Late arrival detected.
EarlyLeave	Check-out before scheduled end for a non-overnight shift.
Overtime	Eligible overtime awarded.
LateOvertime	Both late arrival and overtime are present.
MissingIn	No check-in.
MissingOut	No check-out or invalid CheckOut <= CheckIn.
WEEKLY_OFF / HOLIDAY / LEAVE	Calendar classification statuses stored for non-working days; LEAVE remains the full-day leave classification while a 0.50 leave is written as HalfDay.

6. Leave Integration

LeaveRequest already contains TotalDays. The Attendance change does not modify LeaveRequest, LeaveRequestStatus or LeaveDayPart. AttendanceCalendarBusinessRules now retrieves the approved LeaveRequest rather than only a boolean so the processor can see TotalDays.

Approved status is found by LeaveRequestStatus.Code = APPROVED, with IsActive and !IsDeleted. The leave request must match EmployeeId, cover the date (FromDate <= date <= ToDate), and not be deleted.

Full day: TotalDays 1.00 → LEAVE. **Half day:** TotalDays 0.50 → AttendanceRecord.Status HalfDay. This uses the existing Leave calculation and avoids duplicate leave-day calculations.

7. Manual Attendance Create / Update

CreateManualAttendanceRecord: prevents duplicate employee/date records; resolves assignment, shift and policy; builds the record; invokes the attendance calculation; saves it.

UpdateAttendanceRecord: loads the record; resolves assignment/shift/policy for the attendance date; updates CheckIn, CheckOut and Remarks; invokes AttendanceCalculationService. The old duplicate Recalculate method was replaced by the centralized service.

Persistence check: the current Update handler shown during development does not explicitly call a write repository after calculation. Verify that the underlying tracked entity/save pipeline persists the update in the actual runtime configuration.

8. APIs

Controller	Method & Route	Purpose / Parameters
AttendanceRecordsController	POST /api/attendance-records/process	Body: EmployeeId, FromDate, ToDate. Returns message + createdRecords.
AttendanceRecordsController	GET /api/attendance-records/{id}	Route GUID id. Returns AttendanceRecordDto.
AttendanceRecordsController	GET /api/attendance-records	Query: page, employeeId?, fromDate?, toDate?, status?. Returns PagedResult.
AttendanceRecordsController	POST /api/attendance-records/manual	Body: CreateManualAttendanceRecordCommand. Returns created GUID with 201.
AttendanceRecordsController	PUT /api/attendance-records/{id}	Route GUID id + UpdateAttendanceRecordCommand. Requires route id == command id; returns 204.

Controller convention: the project is being standardized to **[ApiController]** + **[Route("api/[controller]")]** for all controllers.

9. API Processing Example

Example request body:

```
{ "employeeId": "19ed1df1-c3cc-4614-85f0-08def7b598a7", "fromDate": "2026-08-01", "toDate": "2026-08-10" }
```

Example response:

```
{ "message": "Attendance records processed successfully.", "createdRecords": 8 }
```

10. Project / Folder Structure

```
src/ HRMS.API/ Controllers/Attendance/AttendanceRecordsController.cs Middleware/ HRMS.BuildingBlocks/
Application/Abstractions/Persistence/ IReadRepository.cs IWriteRepository.cs Application/Behaviors/
Application/Exceptions/ Application/Pagination/ Modules/Attendance/ Application/
Features/AttendanceRecords/ BusinessRules/AttendanceCalendarBusinessRules.cs
Commands/CreateManualAttendanceRecord/ Commands/UpdateAttendanceRecord/ Commands/ProcessAttendanceRecords/
Queries/GetAttendanceRecordById/ Queries/GetAttendanceRecords/ Services/ AttendanceCalculationService.cs
AttendanceDayStatusService.cs AttendanceDayStatusResult.cs AttendanceDayStatusCodes.cs
IAttendanceCalculationService.cs IAttendanceDayStatusService.cs Domain/Entities/ AttendanceRecord.cs
AttendanceRawLog.cs AttendanceShift.cs AttendancePolicy.cs EmployeeShiftAssignment.cs
AttendanceDayStatus.cs Modules/Leave/ Domain/Entities/ LeaveRequest.cs LeaveRequestStatus.cs
LeaveDayPart.cs HRMS.Persistence/
```

11. Dependencies & Runtime Requirements

Required services include MediatR, generic IReadRepository/IWriteRepository, Attendance domain entities, Leave domain entities for approved leave lookup, persistence/EF infrastructure, and DI registrations for IAttendanceCalculationService and IAttendanceDayStatusService. The API also registers Attendance application/infrastructure services.

Before processing, verify: database connection; repositories resolve; active employee shift assignment; active shift; active policy; AttendanceDayStatuses master rows; APPROVED LeaveRequestStatus where leave exists; CompanyHoliday data where applicable; raw attendance logs for working days.

No migration is required for the half-day enhancement because it uses existing LeaveRequest.TotalDays and existing AttendanceRecord.Status. No HALF_DAY master row is required.

12. Testing Matrix

Test	Expected
Working day with valid punches	Worked/late/early/overtime calculated and final attendance status returned.
No punches on working day	Absent; 'No attendance punches found.'
Saturday/Sunday	WEEKLY_OFF.
Company holiday	HOLIDAY.
Approved full-day leave (1.00)	LEAVE.
Approved half-day leave (0.50)	HalfDay; no punch requirement.
Late arrival	LateMinutes > 0; Late unless a higher-priority combined status applies.

Test	Expected
Early leave	EarlyLeaveMinutes > 0 on non-overnight shift; EarlyLeave unless another higher-priority status applies.
Overtime	OvertimeMinutes awarded only when policy permits and minimum is met; capped at maximum.
Late + overtime	LateOvertime.
Missing check-in	MissingIn.
Missing check-out	MissingOut.
CheckOut <= CheckIn	MissingOut.
Duplicate processing	Existing non-deleted employee/date record is skipped.
Manual update	Derived calculation values are recalculated after CheckIn/CheckOut changes.

13. Verified Development Scenarios

Previously verified records included weekly offs, holiday, absent days, Present, EarlyLeave and Overtime. Manual update testing changed 09:00–18:00 to 09:30–18:30 and produced 480 WorkedMinutes, 0 LateMinutes, 0 EarlyLeaveMinutes, 0 OvertimeMinutes and Present status. Range processing returned createdRecords = 8 in the supplied test.

The approved half-day leave supplied during development has TotalDays = 0.50. After the current code change, the required final regression is to process 2027-04-16 and verify Status = HalfDay, then process a known 1.00-day approved leave and verify Status = LEAVE.

14. Known Boundaries / Future Considerations

The current AttendanceCalendarBusinessRules treats Saturday and Sunday as weekly offs; organization-specific rosters are not part of the current rule. Overnight early-leave behavior is intentionally limited in the current calculation implementation. Attendance status values are currently strings. Half-day leave is Option 1: leave itself determines HalfDay and does not require punch data. More complex partial-day attendance/leave overlap is outside this current stable-scope change.

If future requirements need different behavior for morning vs afternoon half-day leave, partial-day punch validation, payroll proration, or roster-based weekly offs, those should be introduced as explicit business rules with regression tests rather than modifying the current logic ad hoc.

15. Developer Checklist

• Keep calculation formulas only in AttendanceCalculationService.

• Keep calendar classification in AttendanceCalendarBusinessRules / AttendanceDayStatusService.

• Do not recalculate LeaveRequest.TotalDays inside Attendance.

• Keep full-day leave behavior unchanged.

• Use TotalDays = 0.50 as the current half-day leave indicator.

• Do not add HALF_DAY to AttendanceDayStatuses under the current design.

• Run dotnet build after attendance changes.

• Regression-test working day, weekly off, holiday, full-day leave, half-day leave, absent, late, early leave and overtime.

16. Business User Quick View

The system first decides **what kind of day** it is. Weekly off, holiday and leave do not need punch calculation. On working days it checks employee punches and applies the assigned shift and policy. A full-day approved leave is LEAVE. A half-day approved leave is HalfDay because LeaveRequest.TotalDays is 0.50. The Leave module remains responsible for calculating that 0.50 value.

Document status: Updated for the latest stable Attendance design, including the half-day leave rule, detailed calculation formulas, business rules, API reference, project structure, dependencies and testing guidance.